



FLACSO
ECUADOR

SESIÓN 6: SQL — DDL Y DML

Crear, modificar y poblar tablas

Fausto Jácome Pérez
FLACSO Ecuador

Ya sabemos consultar datos. ¿Cómo creamos las estructuras que los contienen y cómo los modificamos?

¿Dónde estamos?

3-4

SESIONES ANTERIORES

ETL con Polars: ingesta, limpieza, joins, reshaping, agrupaciones y window functions sobre archivos locales.

5

SESIÓN ANTERIOR

SQL de lectura: SELECT, WHERE, JOIN, GROUP BY, HAVING y window functions sobre PostgreSQL. Solo lectura sobre `public`.

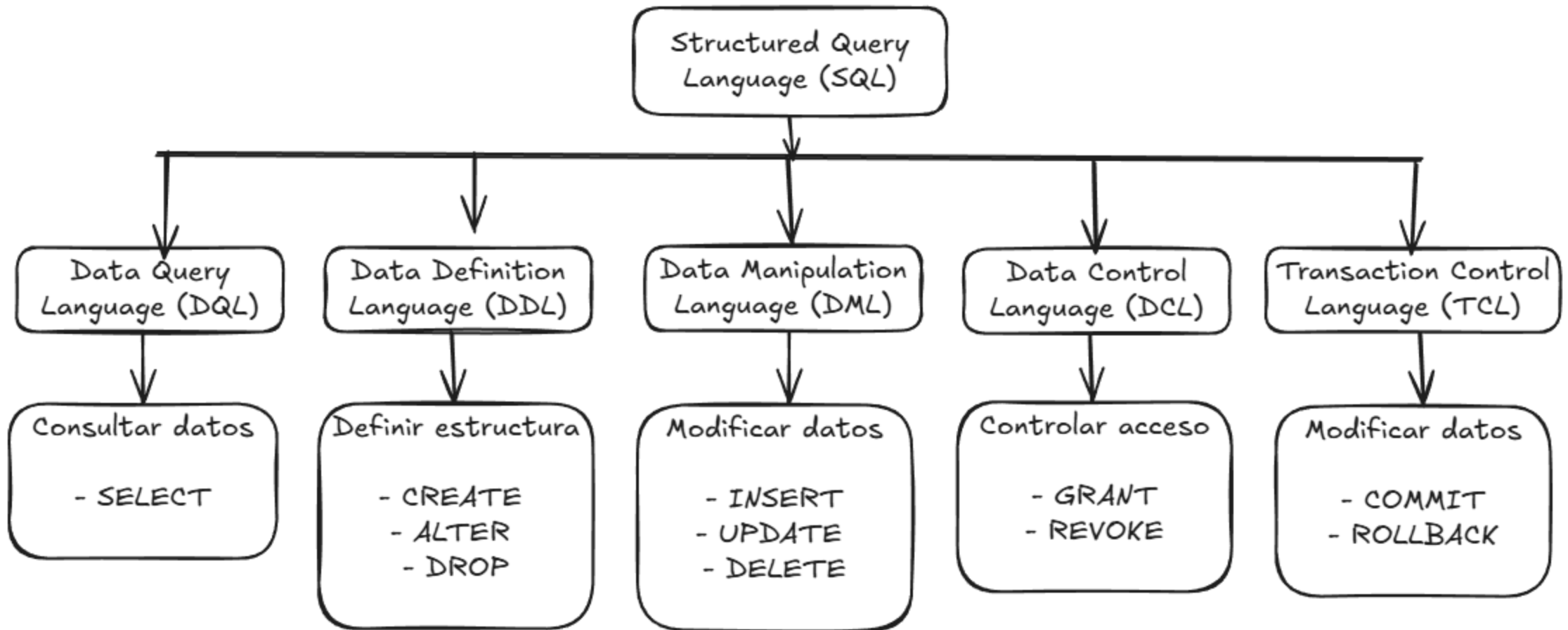
6

ESTA SESIÓN

DDL: crear, modificar y eliminar tablas.
DML: insertar, actualizar y borrar filas.
Cada quien en su propio schema.

Las tablas son las mismas del registro escolar ficticio. Ahora cada alumno tiene un schema personal con permisos completos de escritura.

Las familias de comandos SQL



En la sesión 5 trabajamos solo con DQL. Hoy agregamos DDL y DML. DCL y TCL los veremos de forma aplicada al final.

Schemas y permisos

Por qué cada quien tiene su propio espacio de trabajo

Schemas: espacios de nombres en PostgreSQL

Un schema es un contenedor de objetos dentro de una base de datos. Permite que distintos usuarios tengan tablas con el mismo nombre sin conflicto.

Schema `public`

- Compartido por todos
- Solo lectura para los alumnos
- Contiene las tablas de referencia: `alumnos`, `notas`, `viaje`, etc.

Schema personal `<tu_esquema>`

- Exclusivo de cada alumno
- Permisos completos: `CREATE`, `INSERT`, `UPDATE`, `DELETE`
- Aislado del resto: lo que creas aquí no afecta a nadie más

Intentar escribir sobre `public` produce un error de permisos. El ejercicio 0 lo comprueba directamente.

SET search_path

Sin `search_path` hay que calificar cada nombre de tabla con el schema:

```
SELECT * FROM <tu_esquema>.alumnos;  
ALTER TABLE <tu_esquema>.alumnos ADD COLUMN email TEXT;
```

Con `SET search_path` el motor busca primero en el schema indicado:

```
SET search_path TO <tu_esquema>;  
  
-- A partir de aquí solo el nombre de la tabla  
SELECT * FROM alumnos;  
ALTER TABLE alumnos ADD COLUMN email TEXT;
```

`SET search_path` aplica solo a la sesión activa. Al reconectar hay que ejecutarlo de nuevo. En Colab se hace con `sql(f"SET search_path TO {TU_ESQUEMA};")` justo después de conectar.

DDL: definir estructuras

CREATE TABLE, ALTER TABLE, DROP TABLE

Llaves primarias y foráneas

Antes de crear tablas, dos conceptos que gobiernan la integridad relacional:

Llave primaria (PRIMARY KEY)

- Identifica de forma única cada fila
- No puede ser `NULL` ni duplicada
- Una tabla tiene como máximo una PK
- Puede ser una columna o una combinación

```
cod_estudiante INTEGER PRIMARY KEY
```

Llave foránea (FOREIGN KEY)

- Referencia la PK de otra tabla
- Garantiza que el valor exista en la tabla referenciada
- Puede ser `NULL` (relación opcional)
- Una tabla puede tener múltiples FK

```
cod_estudiante INTEGER  
REFERENCES alumnos (cod_estudiante)
```

Sin estas restricciones es posible registrar notas de alumnos que no existen, o tener dos alumnos con el mismo código. La base de datos no puede detectarlo.

CREATE TABLE

```
CREATE TABLE <tu_esquema>.alumnos (  
  cod_estudiante INTEGER PRIMARY KEY,  
  nombre TEXT NOT NULL,  
  sexo TEXT NOT NULL CHECK (sexo IN ('m', 'f')),  
  edad INTEGER CHECK (edad >= 0 AND edad <= 120),  
  pais TEXT,  
  ciudad TEXT,  
  altura NUMERIC,  
  paralelo TEXT,  
  anio_nac INTEGER,  
  mes_nac INTEGER,  
  dia_nac INTEGER  
);
```

Las restricciones se evalúan en cada `INSERT` y `UPDATE`. Si alguna se viola, la operación falla y la fila no se modifica.

Restricciones (constraints)

Restricción	¿Qué garantiza?
PRIMARY KEY	Unicidad y no nulo en la columna llave
NOT NULL	La columna no puede quedar vacía
CHECK (condición)	El valor cumple una expresión lógica
REFERENCES tabla (col)	El valor existe como PK en otra tabla
UNIQUE	No hay duplicados en la columna

El caso de `viaje.cuotas_pagadas` : esa columna no tiene `CHECK` , por eso encontramos `'si'` , `'SI'` , `'Sí'` , `'pagado'` para el mismo concepto. Un `CHECK (cuotas_pagadas IN ('si', 'no', 'parcial'))` al crear la tabla hubiera forzado consistencia desde el origen.

Restricciones nombradas

Las restricciones pueden definirse al final de la tabla con nombre explícito:

```
CREATE TABLE notas (  
  cod_estudiante INTEGER,  
  nombre TEXT,  
  materia TEXT,  
  nota NUMERIC,  
  CONSTRAINT notas_cod_estudiante_fk FOREIGN KEY (cod_estudiante) REFERENCES alumnos (cod_estudiante) ON DELETE CASCADE,  
  CONSTRAINT notas_cod_estudiante_not_null CHECK (cod_estudiante IS NOT NULL),  
  CONSTRAINT notas_materia_not_null CHECK (materia IS NOT NULL),  
  CONSTRAINT notas_nota_rango CHECK (nota >= 0 AND nota <= 10)  
);
```

El nombre autogenerado se usa en la versión en la que no nombramos la restricción (ver dos diapositivas antes).

FOREIGN KEY y ON DELETE CASCADE

```
CREATE TABLE notas (  
  cod_estudiante INTEGER NOT NULL REFERENCES alumnos (cod_estudiante) ON DELETE CASCADE,  
  nombre TEXT,  
  materia TEXT NOT NULL,  
  nota NUMERIC CHECK (nota >= 0 AND nota <= 10)  
);
```

ON DELETE CASCADE define qué pasa con las filas hijas cuando se borra una fila padre:

Opción	Comportamiento
CASCADE	Borra las filas hijas automáticamente
SET NULL	Pone NULL en la FK de las hijas
RESTRICT	Impide borrar si hay hijas (default)
NO ACTION	Similar a RESTRICT , evaluado al final

ALTER TABLE

Modificar una tabla existente sin recrearla:

```
-- Agregar una columna
ALTER TABLE alumnos ADD COLUMN email TEXT;

-- Eliminar una columna
ALTER TABLE alumnos DROP COLUMN email;

-- Renombrar una columna
ALTER TABLE alumnos RENAME COLUMN anio_nac TO year_nac;

-- Renombrar la tabla
ALTER TABLE notas_123 RENAME TO notas;

-- Cambiar el tipo de una columna
ALTER TABLE alumnos ALTER COLUMN altura TYPE FLOAT;
```

DROP TABLE

```
-- Eliminar una tabla
DROP TABLE notas;

-- IF EXISTS evita error si la tabla no existe
DROP TABLE IF EXISTS notas;

-- CASCADE elimina también los objetos que dependen de ella
-- (vistas, restricciones de otras tablas que la referencian)
DROP TABLE IF EXISTS alumnos CASCADE;
```

`DROP TABLE` elimina la estructura **y todos los datos**. No hay deshacer. En producción siempre verificar dos veces antes de ejecutarlo.

DML: modificar datos

INSERT, UPDATE, DELETE, TRUNCATE

INSERT

-- Fila a fila: especificar columnas explícitamente

```
INSERT INTO alumnos (cod_estudiante, nombre, sexo, edad, pais, ciudad, altura, paralelo, anio_nac, mes_nac, dia_nac)
VALUES (1, 'Ana Torres', 'f', 28, 'Ecuador', 'Quito', 165, 'A', 1996, 3, 12);
```

-- Insertar desde otra tabla con SELECT

```
INSERT INTO alumnos (cod_estudiante, nombre, sexo, edad, pais, ciudad, altura, paralelo, anio_nac, mes_nac, dia_nac)
    SELECT cod_estudiante, nombre, sexo, edad, pais, ciudad, altura, paralelo, anio_nac, mes_nac, dia_nac
    FROM public.alumnos;
```

Siempre listar las columnas explícitamente en el **INSERT** . Sin lista de columnas, PostgreSQL asigna los valores por posición: si la tabla cambia de estructura con un **ALTER TABLE** , los valores pueden quedar en las columnas equivocadas sin ningún error.

UPDATE

```
-- Con WHERE: afecta solo las filas que cumplen la condición
UPDATE alumnos
SET    ciudad = 'Cuenca'
WHERE  cod_estudiante = 1;

-- Actualizar múltiples columnas a la vez
UPDATE alumnos
SET    ciudad    = 'Cuenca',
       paralelo  = 'B'
WHERE  cod_estudiante = 1;
```

Un `UPDATE` sin `WHERE` actualiza **todas las filas** de la tabla. Siempre verificar con un `SELECT` previo que el `WHERE` selecciona las filas correctas.

```
-- Patrón seguro: verificar antes de actualizar
SELECT * FROM alumnos WHERE cod_estudiante = 1;
-- Si el resultado es el esperado, ejecutar el UPDATE
```

DELETE y TRUNCATE

```
-- DELETE con WHERE: elimina filas específicas
DELETE FROM alumnos WHERE cod_estudiante = 500;

-- DELETE sin WHERE: elimina TODAS las filas (con log fila a fila)

-- TRUNCATE: vacía la tabla entera de forma rápida
-- No genera log por fila, no se puede usar con WHERE
TRUNCATE TABLE alumnos;

-- CASCADE necesario si hay FK apuntando a la tabla
TRUNCATE TABLE alumnos CASCADE;
```

	DELETE	TRUNCATE
Familia	DML	DDL
WHERE	Sí	No
Velocidad en tablas grandes	Lenta	Rápida
Se puede revertir con <code>ROLLBACK</code>	Sí	En Postgres sí

Transacciones: BEGIN, COMMIT, ROLLBACK

Las transacciones agrupan operaciones en una unidad atómica: o todas se confirman o ninguna.

```
-- Abrir una transacción explícita
BEGIN;

-- Operación riesgosa
UPDATE alumnos SET paralelo = 'Z';

-- Ver el efecto antes de confirmar
SELECT cod_estudiante, nombre, paralelo FROM alumnos;

-- Si el resultado es correcto: confirmar
COMMIT;

-- Si algo está mal: deshacer todo
ROLLBACK;
```

Mientras no se ejecute **COMMIT**, los cambios son visibles solo en la sesión activa. **ROLLBACK** los deshace completamente como si nunca hubieran ocurrido.

Integridad referencial en acción

Con `ON DELETE CASCADE`, borrar un alumno elimina automáticamente sus notas:

```
-- Notas del alumno 1 antes de borrarlo
SELECT * FROM notas WHERE cod_estudiante = 1;
-- resultado: 2 filas (matematicas y castellano)

-- Borrar el alumno
DELETE FROM alumnos WHERE cod_estudiante = 1;

-- Las notas desaparecieron automáticamente
SELECT * FROM notas WHERE cod_estudiante = 1;
-- resultado: 0 filas
```

Sin `CASCADE`, el `DELETE` sobre `alumnos` fallaría con un error de integridad referencial si existen notas asociadas. La base protege la consistencia.

¿Qué aprendimos hoy?

Conceptos

- Schemas como espacios de nombres y permisos
- `SET search_path` para simplificar las consultas
- Llave primaria y llave foránea
- Restricciones: `NOT NULL`, `CHECK`, `REFERENCES`, `UNIQUE`
- Restricciones nombradas para mensajes de error legibles
- `ON DELETE CASCADE`, `SET NULL`, `RESTRICT`
- Transacciones: `BEGIN`, `COMMIT`, `ROLLBACK`

En SQL / PostgreSQL

- `CREATE TABLE` con constraints
- `ALTER TABLE`: `ADD COLUMN`, `DROP COLUMN`, `RENAME`
- `DROP TABLE IF EXISTS`
- `INSERT` fila a fila e `INSERT ... SELECT`
- `UPDATE` con y sin `WHERE`
- `DELETE` con y sin `WHERE`
- `TRUNCATE TABLE ... CASCADE`
- Integridad referencial en acción con `CASCADE`

Extra: ya entienden los memes de SQL

**YOU USE
SQL
UPDATE**



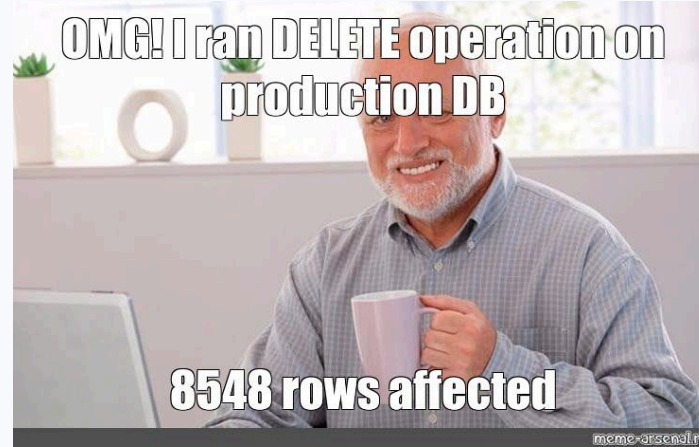
**YOU FORGET
SQL WHERE**

**37,052 ROWS
AFFECTED**



**OMG! I ran DELETE operation on
production DB**

8548 rows affected



Query OK, 14882915 rows affected (39.24 sec)

